Fork Progress Report

CaseMirror Research

casemirror.ai

June 6, 2026

Project: `sia_rust`

Repository: `/home/m/sia_rust`

Survey date: 2026-06-06

Survey HEAD: `da4191a` (`main`)

A. Executive Summary

Since the fork from the upstream SIA project, this repository has been transformed

from a Python-first self-improving-agent framework into an active Rust port with a

Python execution bridge, native Rust meta/feedback LLM runners behind an optional

`llm` feature, parity tests, a file-backed web dashboard, benchmark/eval harnesses,

and a substantial demo/reproducibility documentation package.

The fork-era work is concentrated in 48 commits on 2026-06-06 after the inferred

upstream baseline `4b4877f` ('chore: bump minor version as cli contracts have

changed (#23)'). Across that range, the branch changes 139 files with 33,895

insertions and 333 deletions. The biggest areas of change are `src/`, `src/llm/`,

`tests/`, `docs/`, `benchmarks/`, `evals/`, and the project issue-tracking state

under `.beads/`.

The project is in a strong Rust-port state for deterministic/offline surfaces:

default Rust tests pass, default and `llm` clippy pass, the standalone `evals/`

crate passes, and the Python/Rust differential parity script reports byte-identical

output on all checked surfaces. The main remaining risks are live-demo readiness,

live provider validation, hardening of the runtime security story, and replacing

some still-observational or reference-only "closed loop" pieces with acting,

production-quality behavior.

B. Scope And Baseline

There is no `upstream` remote configured in this checkout. The only configured

remote is:

- `origin`: `git@github.com:micahstubbs/sia_rust.git`

Accordingly, this report treats the last pre-fork upstream-looking commit as an

inferred baseline rather than a remote-confirmed fork point:

- Upstream baseline used for this report: `4b4877f`
- Baseline subject: `chore: bump minor version as cli contracts have changed (23)`
- Commits through that baseline: 11
- Fork-era commits after that baseline at survey time: 48

At survey time, `main` was ahead of `origin/main` by one commit:

- `da4191a` - `Sync beads tracker after upstream review`

During PDF generation, `origin/main` advanced by three additional remote commits

(`18e2f24`, `9442ced`, and `f1890b1`). This report evaluates the local checkout at

`da4191a` and does not include those later remote commits.

The report/PDF artifact commit generated from this document is not included in

those counts.

C. Work Completed Since The Fork

1. Rust Port Foundation

The largest milestone is `49415c2`, 'Rust port of SIA framework (exhaustive parity

+ benchmarks + dsrs evals)'. It introduced the Rust crate, CLI, CI workflow,

benchmarks, tests, parity tooling, and the core ported modules.

Major Rust modules now mirror the Python package surface:

- Configuration, providers, profiles, API-key resolution, and bundled config files.
- Task layout and run-directory setup.
- Prompt building and context management.
- Orchestrator and generation loop scaffolding.
- Target-agent execution through a Python subprocess using the existing

`evaluate.py` contract.

- Web data model and Axum server for `sia web`.
- CPython-compatible JSON formatting for byte-for-byte parity.

The public project framing was also rewritten. `README.md` now presents this as a

Rust port of SIA with a Python bridge rather than as the original Python package.

`docs/RUST_PORT.md` documents the Python-to-Rust module map, build/test commands,

native-runner feature gate, parity surfaces, benchmarks, evals, and known testing

seams.

2. Native Meta/Feedback LLM Runners

The fork added a substantial native LLM layer under `src/llm/`, gated by the

optional `llm` Cargo feature. This includes:

- `AgentRunner` abstraction and trajectory context/outcome types.
- Anthropic Messages API transport and OpenAI-compatible chat transport types.
- Claude, OpenHands-style, and PydanticAI-style runner loops.
- Tool execution for read/write/edit/glob/bash.
- Trajectory logging middleware.
- Retry/backoff and transport decorators.
- Structured-output extraction/parity support.
- Provider/profile-to-client mapping.
- Per-run telemetry generation.
- Tavily web-search client primitive.

This closes a major boundary from the initial port: meta/feedback agents can now

use native Rust LLM clients when the crate is built with `--features llm`, while the target agent remains a Python subprocess so generated task agents preserve the

paper's task/evaluator contract.

3. Closed-Loop Extensions: Scheduler, Weights, And Verifiers

The fork adds first-pass closed-loop machinery beyond the baseline SIA harness

update flow:

- Adaptive scheduler logic in `src/scheduler.rs`.
- Scheduler decision recording and dashboard surfacing through `src/closed_loop.rs`

and web data endpoints.

- A native `WeightUpdater` abstraction and CPU reference LoRA-style updater in

`src/weights.rs`.

- Reusable verifier traits and task robustness helpers in `src/verifier.rs`.

These are meaningful research/product extensions, but the issue tracker correctly

records that some are still partial. In particular, the current scheduler path is

observational in key places, and the CPU weight-update path is a reference toy

rather than real model training or an adapter that is loaded by the target agent.

4. SIA Studio And Web/Demo Surface

The fork upgraded the web surface into a more complete "SIA Studio" dashboard:

- Telemetry and metrics charts.
- Dark-mode/polished UI work.
- Run-level scheduler decision timeline.
- API endpoints for telemetry, metrics, scheduler decisions, and weight updates.

The dashboard is functional for file-backed run visualization, but current open

issues identify important demo gaps: auto-refresh is missing, port-bind failures

can be hidden, and run IDs can collide during rehearsal or stage demos.

5. Security And Sandboxing

The fork added a formal security posture and first hardening primitives:

- `SECURITY.md` threat model.

- src/sandbox.rs capability allow-list abstraction.
- Tests for read/write/bash permissions, path containment, and size limits.
- Native tool executors wired to capability checks.
- Docker sandbox command generation for Python target-agent execution.

The current security story is honest but not finished. The default native-runner

capability profile still allows broad Bash behavior, and the Docker sandbox path is

not yet a drop-in safe path for live LLM-calling target agents because network,

dependencies, and credentials require an explicit policy.

6. Providers, Credentials, And Reproducibility

The fork added or expanded provider support and documentation:

- Bundled Nebius profiles and model-slug verification tooling.
- Bundled Gemini target profile and tests.
- Bundled Tinker provider plus gptoss-tinker-target and

qwen3-tinker-target profiles.

- .env loading at startup, with real environment variables taking precedence.
- Per-provider credentials documentation in docs/CREDENTIALS.md.
- Nebius quickstart and model verification docs.
- Reproducibility standards and run-artifact checklist.
- Hackathon demo script, slide outline, run-of-show, and judging narrative.
- Preprint-style draft in docs/paper/sia_rust_preprint.md.

This gives the project a strong external-facing package, but live provider paths

remain mostly guarded behind ignored tests that require real keys and network

access.

7. Benchmarks And Evals

The fork added multiple validation/evaluation assets:

- Criterion benchmark scaffold under benches/core.rs.
- Python-vs-Rust benchmark scripts under benchmarks/.
- benchmarks/REPORT.md, documenting the deterministic core as about 5.8x faster

by geometric mean in the benchmark report.

- Standalone evals/ crate using DSRs/dspy-rs for a GPQA-style multiple-choice

evaluation harness with offline mock tests.

- Differential parity script at scripts/parity_check.py.

8. Project Operations And Issue Tracking

The latest local commits added project operations scaffolding:

- .beads/ project configuration and issue export.
- CLAUDE.md local agent guidance.
- package.json.
- scripts/seed_beads_from_gh.sh.
- A session summary under docs/session-summaries/.
- Mirroring of 18 open GitHub issues into Beads.
- Subsequent Beads synchronization after upstream review.
- Expanded CLAUDE.md local agent instructions covering build, test, parity, and

architecture guidance.

- Python packaging metadata guard for the OpenHands optional extra, constraining

it to supported Python versions.

At survey time, br list showed 18 open issues. br ready --limit 20 showed 17

ready issues with no blockers. The P1 ready work is heavily demo/safety oriented:

- Harden native-runner capability profile for demo safety.
- Make Docker sandbox usable or clearly scoped for LLM-calling target agents.
- Add a stage-safe live demo command.
- Make SIA Studio auto-refresh during live runs.

D. Current Verification Results

Commands run during this survey:

- Default Rust suite: passed with cargo test; the library crate reported 154

passed tests, and the integration/doc tests also passed.

- Parallel llm suite: failed under default parallel execution; the library

crate reported 272 passed, 4 failed, and 6 ignored tests before aborting. The

failures were llm::provider_mapping tests racing on process environment

variables.

- Serialized `llm` suite: passed with `--test-threads=1`; the library crate

reported 276 passed and 6 ignored tests, and the `llm` integration/doc tests

also passed.

- Standalone eval crate: passed with 4 offline tests.
- Formatting and lints: `cargo fmt --check`, default clippy, and `llm` clippy all

passed with warnings denied.

- Differential parity: passed; the script reported 'PARITY OK: all surfaces

byte-identical'.

- Python pytest suite: not run because `pytest` is not installed in this

environment.

- Packaging metadata unittest: passed.

The normal parallel `llm` test failure is worth treating as a real test-isolation

issue even though serialized execution passes. The failing tests mutate global

environment variables using a helper that snapshots/restores state, but other

parallel tests also read or mutate those variables. A mutex or serial-test pattern

would make this robust under default `cargo test --features llm`.

E. Current Worktree State

At survey time, before updating this document and generating the PDF artifacts,

the worktree was clean. `main` was one commit ahead of `origin/main`, with the

only ahead commit being `da4191a` (`Sync beads tracker after upstream review`).

During report/PDF generation, an unstaged `.beads/issues.jsonl` update appeared;

it is intentionally outside the report artifact commit.

F. Main Remaining Risks

The current backlog and verification results point to these high-value next steps:

1. Fix the `llm` feature test race so `cargo test --features llm` passes under

default parallel test execution.

1. Run live provider validation with real keys: ignored Anthropic, OpenHands,

PydanticAI, Gemini, Nebius, structured-output, and Tavily live tests.

1. Run a complete live `sia run --features llm` self-improvement loop and capture

the artifacts.

1. Make the scheduler decision actually drive the generation loop rather than only

producing artifacts/logging.

1. Replace or supplement the CPU reference weight updater with a real model update

path, or clearly scope it as an offline demonstrator.

1. Harden native-runner tool permissions for demo mode.
2. Clarify or implement Docker sandbox support for live provider-calling target

agents.

1. Fix demo fragility: SIA Studio auto-refresh, dashboard port handling,

collision-proof run IDs, and a fast stage-safe task.

1. Resolve documentation/package identity issues that still point to the original

Python repository.

1. Install/use the Python test environment so the full Python pytest suite can be

included in regular verification.

G. Overall Assessment

The fork has already accomplished the hard architectural work: the deterministic

SIA loop is represented in Rust, parity is actively tested, native meta/feedback

LLM runners exist, telemetry and dashboard surfaces are in place, and the project

has a strong documentation and demo package.

The project is not yet "live-demo hardened." Its deterministic and offline

verification story is strong, but the live-provider path, security defaults,

dashboard freshness, and closed-loop actuation still need focused work before the

repository can credibly claim a fully production-ready self-improving loop.